

Dispositivos Conectados

Sensores/atuadores SPI

2025/2026

Authors: Paulo C. Bartolomeu

1 Objetivos

- Configuração e utilização de um barramento SPI;
- Leitura e escrita num cartão microSD através do barramento SPI;
- Integração do multi-sensor DHT20 para leitura de temperatura e humidade e respetivo *logging* num ficheiro do cartão microSD.

2 Introdução

O barramento SPI (*Serial Peripheral Interface*) é essencial no ecossistema de sistemas embebidos, permitindo a comunicação rápida e eficiente entre um microcontrolador e uma grande diversidade de periféricos — desde sensores e atuadores até módulos de comunicação e dispositivos de armazenamento. A sua simplicidade estrutural, baseada em poucas linhas de sinal, torna-o particularmente adequado para integrar múltiplos componentes num único sistema, mantendo baixos custos e elevada fiabilidade.

No contexto de plataformas como ESP32-C6, o SPI é frequentemente utilizado para ligar sensores de medição, controladores de interface, módulos de expansão e cartões de memória. Esta versatilidade torna-o uma ferramenta estratégica para o desenvolvimento de aplicações de IoT e automação, onde é comum combinar aquisição de dados, processamento local e armazenamento.

A utilização de cartões SD através de SPI acrescenta uma camada importante de funcionalidade: o armazenamento local persistente. Isto permite, por exemplo, guardar registos de sensores, eventos de operação ou configurações do sistema, mesmo em ambientes sem conectividade. Além disso, o acesso via SPI simplifica a integração de soluções de logging e armazenamento em projetos de investigação ou desenvolvimento industrial, sem a necessidade de hardware complexo adicional.

Dominar a utilização do SPI, incluindo operações de leitura e escrita em cartões SD, é assim um passo fundamental para construir sistemas embebidos robustos, escaláveis e autónomos, capazes de operar de forma eficiente e confiável em cenários reais.

Nesta aula iremos explorar a utilização do protocolo SPI para ler e escrever informação num cartão SD.

3 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base na [API de Referência para os periféricos do ESP32-C6](#) juntamente com o [ESP32-C6 Technical Reference Manual](#). Adicionalmente, **deverão** ser consultadas as fichas técnicas do sensor DHT20 e do *display* Adafruit 160x80 0.96" disponíveis na página do elearning na pasta Recursos.

3.1 Ligação do *display* com leitor de cartões integrado ao esp32-c6 devkit

O primeiro passo para a realização deste trabalho prático consiste na ligação elétrica do *display* ao esp32-c6 devkit. Como pode consultar na ficha técnica do *display*, este incorpora um leitor de cartões SD que será fundamental para esta aula prática. É possível visualizar o TFT e o leitor de cartões nas Figuras 1 e 2, estando disponível informação do *pinout* de ligação na camada *silkscreen* da PCB.

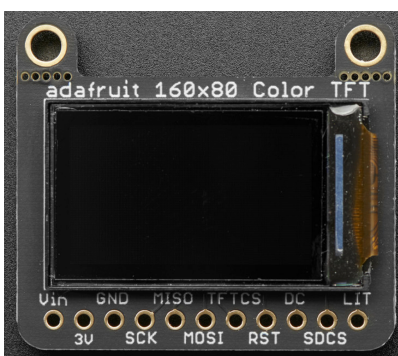


Figura 1: Display Adafruit (Vista do TFT)

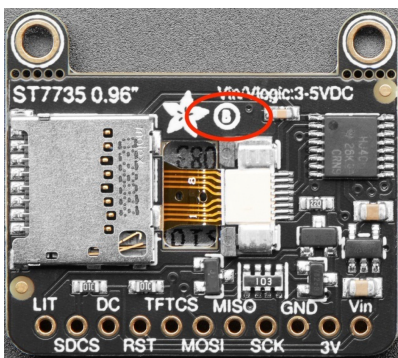


Figura 2: Display Adafruit (vista do leitor de cartões SD)

Como se pode observar, este *display* possui um conjunto de pinos que lhe permite operar simultaneamente como display e como leitor de cartões SD. Os pinos disponíveis para ligação e respetivas funções resumem-se a:

- **3–5 V / Vin:** pino de alimentação. Deve ser ligado a 3–5 V DC. Tem proteção contra polaridade inversa, mas deve ser ligado corretamente.
- **3.3 V:** saída de 3,3 V proveniente do regulador de tensão integrado.
- **GND:** pino de massa, comum à alimentação e aos sinais.
- **SCK:** pino de entrada do relógio SPI. Aceita níveis lógicos de 3–5 V.

- **MISO:** pino *Microcontroller In Serial Out*. Usado pelo cartão SD. Não é utilizado pelo ecrã TFT (apenas escrita). Saída lógica a 3,3 V (compatível com leitura a 5 V).
- **MOSI:** pino *Microcontroller Out Serial In*. Usado para enviar dados do microcontrolador para o cartão SD e/ou ecrã TFT. Aceita níveis lógicos de 3–5 V.
- **TFT_CS:** pino de chip select SPI do ecrã TFT. Aceita níveis lógicos de 3–5 V.
- **RST:** pino de reset do ecrã TFT. Ligar à massa para reiniciar o ecrã. Inclui um circuito de *reset* automático ao ligar a alimentação. Aceita níveis lógicos de 3–5 V.
- **D/C:** pino seletor de dados ou comandos SPI do ecrã TFT. Aceita níveis lógicos de 3–5 V.
- **D/C** — pino seletor de dados ou comandos SPI do ecrã TFT. Aceita níveis lógicos de 3–5 V.
- **SD Card CS / SDCS:** este é o pino de chip select do cartão SD, usado quando se pretende ler dados do cartão. Aceita níveis lógicos de 3–5 V.
- **Lite:** este é o pino de entrada PWM para controlo da retroiluminação. Por defeito está em nível alto (retroiluminação ligada). Pode ser controlado por PWM a qualquer frequência ou colocado em nível baixo para desligar a retroiluminação. Aceita níveis lógicos de 3–5 V.

Neste laboratório iremos usar apenas a funcionalidade de leitor de cartões SD. Por este motivo, **o pino TFT_CS deve ser ligado a 3.3V de modo a que o display permaneça inativo durante os diferentes exercícios**. A Figura 3 mostra as ligações necessárias a habilitar o leitor de cartões no esp32-c6 devkit. Proceda à montagem do circuito, tendo atenção redobrada às ligações efetuadas. Note as **resistências de pull-up** nos sinais MISO, MOSI e SDCS.

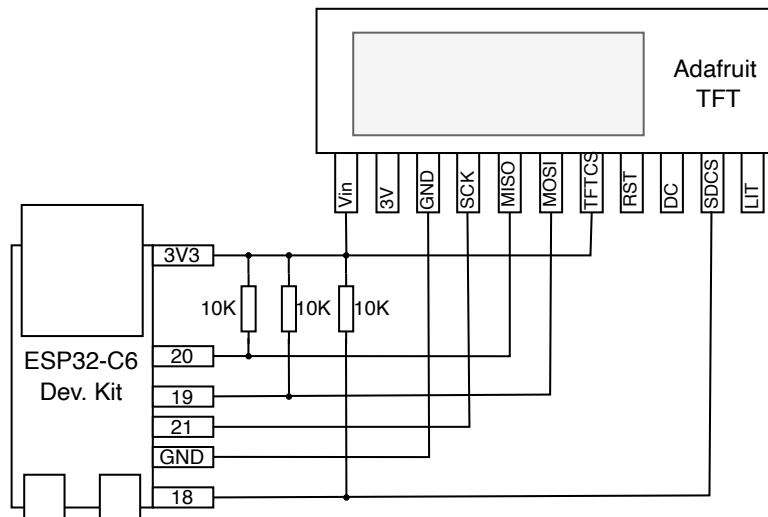


Figura 3: Circuito de ligação SPI ao leitor de cartões

Após a montagem do circuito insira o cartão microSD na ranhura do leitor.

3.2 Aplicação de teste para leitura/escrita de cartões SD

Através do *Wizard* da extensão ESP-IDF, crie um novo projeto tendo como base o exemplo `storage -> sd_card -> sdspi`. De seguida, compile o exemplo e descarregue-o para a placa.

Deverá observar no terminal a seguinte informação:

```

I (444) main_task: Calling app_main()
I (444) example: Initializing SD card
I (444) example: Using SPI peripheral
I (454) example: Mounting filesystem
I (454) sdspl_transaction: cmd=52, R1 response: command not supported
I (504) sdspl_transaction: cmd=5, R1 response: command not supported
I (524) example: Filesystem mounted
Name: 00000
Type: SDHC
Speed: 20.00 MHz (limit: 20.00 MHz)
Size: 29820MB
CSD: ver=2, sector_size=512, capacity=61071360 read_bl_len=9
SSR: bus_width=1
I (524) example: Opening file /sdcard/hello.txt
I (544) example: File written
I (544) example: Renaming file /sdcard/hello.txt to /sdcard/foo.txt
I (554) example: Reading file /sdcard/foo.txt
I (554) example: Read from file: 'Hello 00000!'
I (554) example: Opening file /sdcard/nihao.txt
I (574) example: File written
I (574) example: Reading file /sdcard/nihao.txt
I (574) example: Read from file: 'Nihao 00000!'
I (574) example: Card unmounted
I (574) main_task: Returned from app_main()

```

IMPORTANTE!: A execução do programa será diferente (será mostrado um erro) caso não tenha configurado corretamente os pinos usados no SPI. Para corrigir o erro, deve aceder ao SDK Configuration Editor, procurar a secção SD SPI Example Configuration e atribuir os números dos pinos de forma consistente com o circuito montado.

QUESTÃO 1: Estude o driver SD ([SD SPI Host Driver](#)) e o suporte para o sistema de ficheiros ([FAT Filesystem Support](#)). Analise a construção do código e explique qual é o objetivo deste exemplo, identificando os vários passos realizados.

QUESTÃO 2: Desligue o esp32-c6 devkit da alimentação e retire o cartão microSD. Usando um leitor de cartões SD inspeccione o seu conteúdo. Em que medida reflete a execução do programa?

3.3 Aplicação de log para escrita no cartão SD

Com base no exemplo anterior, pretende-se agora desenvolver algumas variantes do programa, mas com funcionalidades mais úteis.

QUESTÃO 3: Altere o programa anterior para escrever para o ficheiro `log.txt`, que deverá ser guardado na raiz do cartão microSD, o valor em milisegundos desde o arranque da placa juntamente com um valor de contagem usando o formato csv. A escrita no cartão deverá ser realizada com um período de aproximadamente 1 segundo.

Execute o programa, desligue a placa e remova o cartão microSD, lendo o seu conteúdo no portátil. Deverá observar informação semelhante à apresentada em seguida:

```

millis,counter
270,0
1340,1
2400,2
3460,3
4520,4
5580,5
6640,6

```

3.4 Aplicação de leitura do log escrito no cartão SD

Pretende-se agora evoluir o programa anterior para, ao invés de escrever informação para um ficheiro de log, ler essa informação do ficheiro, linha a linha e imprimir os conteúdos na consola.

QUESTÃO 4: Desenvolva um programa que, a cada segundo, realize uma nova leitura de uma linha do ficheiro `log.txt` (armazenado na raiz do cartão microSD) e imprima o seu conteúdo para o terminal. Implemente a funcionalidade de leitura numa tarefa FreeRTOS.

O resultado da leitura deverá ser semelhante ao apresentado em seguida. Note que no processo de leitura, **o cabeçalho do ficheiro é ignorado**.

```
I (446) main_task: Calling app_main()
I (446) sdread: Mounting /sdcard
I (456) sdspi_transaction: cmd=52, R1 response: command not supported
I (496) sdspi_transaction: cmd=5, R1 response: command not supported
I (516) sdread: Mounted. Card: 00000
I (516) main_task: Returned from app_main()
I (516) sdread: CSV: 210,0
I (1516) sdread: CSV: 1250,1
I (2516) sdread: CSV: 2260,2
I (3516) sdread: CSV: 3280,3
I (4516) sdread: CSV: 4300,4
I (5516) sdread: CSV: 5320,5
I (6516) sdread: CSV: 6340,6
I (7516) sdread: CSV: 7360,7
I (8516) sdread: CSV: 8380,8
```

3.5 Integração da leitura da temperatura e humidade (DHT20) com logging no cartão microSD

Finalmente, pretende-se integrar funcionalidades. Neste sentido deverá montar o circuito da Figura 4 que integra o leitor de cartões e o sensor de temperatura e humidade DHT20 usado na aula anterior.

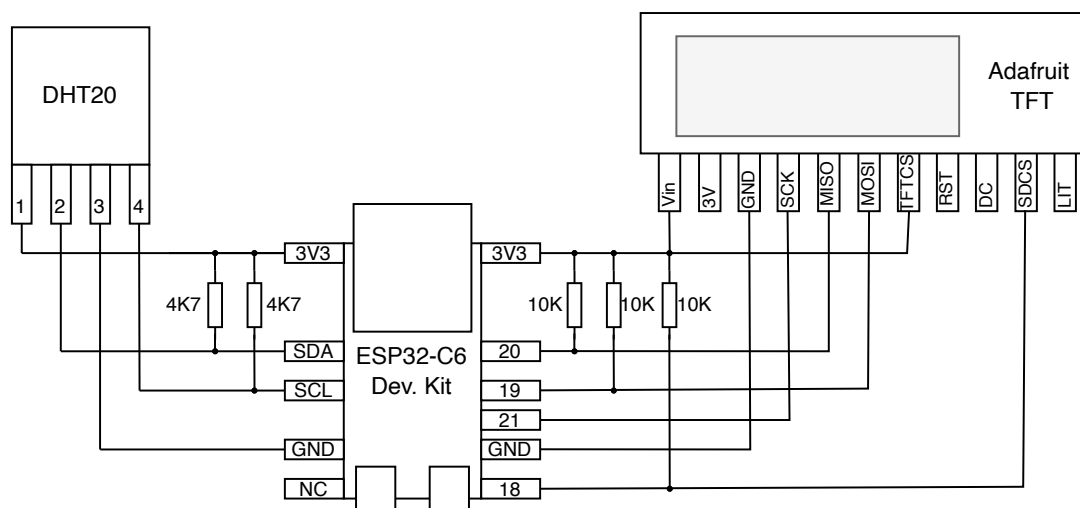


Figura 4: Circuito para integração do leitor de cartões com o sensor DHT20

QUESTÃO 5: Desenvolva um programa que, usando o sensor DHT20 realize a cada segundo uma leitura da temperatura e humidade, salvaguardando esta informação no ficheiro `log.txt` (armazenado na raíz do cartão microSD). A funcionalidade de leitura e escrita no cartão deve ser implementada numa tarefa FreeRTOS. A cada escrita no cartão deve ser impresso na consola informação sobre o tempo decorrido desde a energização da placa, o valor de contagem, a temperatura e a humidade lidas.

Teste o seu programa na placa, devendo obter um resultado semelhante ao seguinte na consola:

```
I (350) sdlog: Mounted. Card: 00000
I (350) sdlog: I2C initialized successfully for DHT20
I (350) main_task: Returned from app_main()
I (440) sdlog: Logged data: time=170 ms, counter=0, temp=23.71 C, hum=74.96 %
I (1500) sdlog: Logged data: time=1230 ms, counter=1, temp=23.72 C, hum=74.94 %
I (2560) sdlog: Logged data: time=2290 ms, counter=2, temp=23.73 C, hum=74.91 %
I (3620) sdlog: Logged data: time=3350 ms, counter=3, temp=23.73 C, hum=74.86 %
I (4680) sdlog: Logged data: time=4410 ms, counter=4, temp=23.71 C, hum=74.91 %
I (5740) sdlog: Logged data: time=5470 ms, counter=5, temp=23.71 C, hum=74.90 %
```

QUESTÃO 6: Desligue a placa e remova o cartão microSD. Consulte o conteúdo do ficheiro `log.txt`, que deverá ser parecido com o seguinte:

```
millis,counter, temperature, humidity
270,0, 23.69, 74.71
1340,1, 23.68, 74.77
2400,2, 23.69, 74.84
3460,3, 23.69, 74.89
4520,4, 23.69, 74.92
5580,5, 23.71, 74.95
```

Nota: Os exemplos mostrados acima não estão consistentes em termos de informação, visto terem sido recolhidos em momentos distintos. Servem apenas como ilustração pedagógica sobre o formato da informação a obter.

4 Acknowledgements

Originally authored by [Paulo C. Bartolomeu](#)